

Hardware Sentinel: Protecting Software Applications from Hardware Silent Data Corruptions

Rhea Dutta
Meta Platforms Inc.
Menlo Park, CA, USA
rheadutta@meta.com

Harish Dattatraya Dixit
Meta Platforms Inc.
Menlo Park, CA, USA
hdd@meta.com

Rik Van Riel
Meta Platforms Inc.
Menlo Park, CA, USA
riel@surriel.com

Gautham Vunnam
Meta Platforms Inc.
Menlo Park, CA, USA
gauthamvunnam@meta.com

Sriram Sankar
Meta Platforms Inc.
Menlo Park, CA, USA
sriramsankar@meta.com

Abstract

Silent Data Corruptions (SDCs) pose a significant challenge in large-scale infrastructures, affecting data center applications unpredictably and reducing service reliability. Primarily caused by silicon defects, traditional hardware testing methods are insufficient to prevent SDC propagation. SDCs are influenced by various factors, including data randomization, workload characteristics, environmental conditions, and aging, necessitating top-down approaches from the application layer. In this paper, we introduce Hardware Sentinel, a novel framework that detects SDCs through *typical software failure indicators* such as segmentation faults, core dumps, application crashes, and logs. We have validated our framework in a large-scale data center fleet, across diverse application, kernel, and hardware configurations, achieving a high success rate of SDC detection. Hardware Sentinel has uncovered novel instances of SDCs, surpassing the detection capabilities of published testing techniques. Our analysis of over 6 years' worth of application and system failure data within a large-scale infrastructure has successfully identified hundreds of defective CPUs that triggered SDCs. Notably, the Hardware Sentinel flow increases effective coverage over existing hardware-testing methods like Fleetscanner (out-of-production testing) by 1.74x and Ripple (in-production testing) by 1.92x. We share the top kernel exceptions with the highest correlation to silent data corruption failures. We present results spanning 7 CPU generations from multiple semiconductor manufacturers, 13 large-scale workloads, and 27 data center regions, providing insights into the trade-offs involved in detection and fleet deployment.

ACM Reference Format:

Rhea Dutta, Harish Dattatraya Dixit, Rik Van Riel, Gautham Vunnam, and Sriram Sankar. 2025. Hardware Sentinel: Protecting Software Applications from Hardware Silent Data Corruptions. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*, March 30-April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3676641.3716258>

1 Introduction

Large-scale online services are hosted on millions of servers across multiple data centers worldwide [12] [15]. Ensuring computational correctness throughout the application stack is crucial for maintaining reliability. However, with increasing transistor density and accelerated manufacturing, computational faults can occur. Central Processing Units (CPUs) can produce incorrect values (e.g., $1+1 = 3$) used by services without immediate error, known as Silent Data Corruptions (SDCs). SDCs are a significant concern in large server fleets, occurring at problematic rates and causing production issues. They can arise from silicon design, manufacturing, environmental conditions, or aging processes. SDCs can silently propagate from hardware to applications, leading to significant engineering resources being spent by application developers and service providers to debug these elusive errors [31].

Meta highlighted issues with Silent Data Corruptions (SDCs) when a machine miscalculated a file size, leading to database errors and application failures [31]. Although SDCs are not a novel phenomenon [42], [36], [14], [17], they were previously considered rare, occurring in approximately *one* defect per *one million* devices [16]. Few institutions or hyperscalers had the scale to validate the occurrence rates of SDCs through extensive testing. The problem space shifted when Meta provided an observed measurement of *one SDC* per *1000* devices [31], nearly three orders of magnitude higher than any previously reported number with soft error induced single event upsets [16]. Meta shared multiple hardware-testing



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '25, March 30-April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1079-7/25/03.

<https://doi.org/10.1145/3676641.3716258>

mechanisms to identify SDCs in the server fleet [30]. Subsequent papers by Google [45], [23] and Alibaba [100] also validated similar rates of SDC occurrence in their server fleets, establishing SDCs as an emerging concern in hyperscaler fleets. Industry consortiums like Open Compute Project [7] have established workstreams [6] to address this critical challenge in server systems.

A common approach to mitigating SDCs in large fleets is hardware-testing, which involves screening hardware before and after deployment, as reported by Meta [31] and Alibaba [100]. However, these techniques rely heavily on vendor-specific knowledge and tools, making it challenging for academic settings to replicate them. Moreover, hardware testing has limitations in diagnosing new SDC signatures, with detection rates affected by various factors such as data randomization, environmental conditions, testing sequences, workload co-location, and aging. Furthermore, both out-of-production and in-production testing require significant coordination across hardware and software layers, adding complexity to the process. To address these gaps, we developed Hardware Sentinel (HWSentinel), a top-down application-based detection framework that complements existing methods. HWSentinel's vendor-agnostic approach enables research teams to independently develop SDC detection methods.

In this paper, we introduce the HWSentinel framework, which leverages six years of diagnosis and remediation data from a large server fleet to detect Silent Data Corruptions (SDCs). HWSentinel employs a novel approach that correlates and reverse engineers SDCs from extensive kernel anomalies and application signatures. Previously, undetected SDCs were often misattributed to software issues or lost in application fault noise. By utilizing HWSentinel, we successfully identified hundreds of failure candidates across various architectures and workloads, which were subsequently validated by independent silicon failure analysis. To the best of our knowledge, HWSentinel is the first application-level SDC detection framework differing from traditional hardware-focused approaches. Our key contributions are as follows:

1. We present HWSentinel, a novel framework for SDC detection that uses application information like System logs, Kernel crashes and OS telemetry. We do not use proprietary hardware information from manufacturers in our approach, and hence HWSentinel is vendor-agnostic.
2. We provide the internal details of the data and control plane implementation, our logging structure, as well as our framework for candidate selection to enable SDC detection in a large fleet. Unlike previous works that summarized SDC observations, we offer an in-depth examination of our framework's implementation.
3. We identify specific exception types which showcase a higher correlation to hardware silent data corruptions, providing valuable insights into SDC detection.
4. We present results spanning 7 CPU generations from multiple semiconductor manufacturers, 13 large-scale workloads, and 27 data

- center regions while offering insights into the trade-offs involved in detection and fleet deployment.
5. We validate our framework with an independent root-cause and failure analysis [98], demonstrating the effectiveness of our detection approach.
6. We compare HWSentinel to existing testing methodologies in literature like Fleetscanner and Ripple [30] as a baseline and show that HWSentinel results in 1.74x improvement in failure detection compared to Fleetscanner (out-of-production testing), 1.92x improvement when compared to Ripple (in-production testing).
7. Furthermore, we highlight additional benefits of HWSentinel for fleet-deployment events like kernel rollouts, achieving a 21% improvement in rollout success rates.

The paper is structured as follows - Section 2 provides a brief overview of application faults and exceptions at scale. Section 3 describes the experimental setup for HWSentinel framework. Section 4 presents the HWSentinel solution, including the data plane and control plane details. Section 5 discusses the results from HWSentinel and offers insights into the findings. Section 6 examines related work focusing on solutions deployed at large-scale server fleets. Section 7 concludes the paper.

2 Application Failures

2.1 Workload Exceptions

In large-scale infrastructures, applications run across globally distributed data centers [12], [15]. These applications consist of binaries on multiple server sets based on workload reservations, regional allocations [54], and availability pools [79]. Our objective is to ensure applications execute workloads without performance and reliability issues. Application targets include performance [91], reliability, query throughput, data availability, and operational latency [95]. These concepts are extensively discussed in the literature on distributed systems [28], [89], [71], [50].

A large-scale fleet comprises distributed binaries executing across numerous servers, where various issues can arise, encountering warnings, Out-Of-Memory (OoM) events, segmentation faults, kernel oops [41], machine check exceptions, Non-maskable Interrupts (NMI), corrected and uncorrected events [86], stalls, and lockups. These are categorized as application exceptions. While some exceptions stem from software and configuration errors, others are hardware-related. All exceptions are aggregated in large-scale databases for analysis providing valuable insights in fleet contexts. When developing applications or testing new kernels, exceptions offer crucial information about program faults. A typical program may execute numerous system calls and interrupts, varying across CPU architectures, and faults can arise from developer errors or system faults [77].

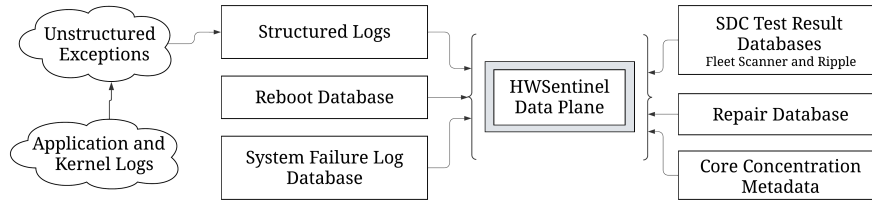


Figure 1. HWSentinel Data Plane

2.2 System Calls and Interrupts

Software applications in a Linux-like environment execute within user space and request kernel space services via system calls [3] and asynchronously through interrupts [1]. Exceptions, including traps, aborts, and faults, are handled by the kernel based on their type [41].

In our fleet, we monitor all application entry points into kernel space that result in an *unsafe* exit, indicating instability. Such exits can occur due to abnormal termination of system calls or stalls resulting from serviced interrupts, ultimately affecting system availability. However, attributing instability to software or hardware faults is a complex task. Fortunately, CPUs provide valuable hints on potential hardware root causes through health telemetry [24].

2.3 SDC Signatures

In SDC-related cases, the CPU cannot provide insight into the root cause, but unsafe exits due to SDCs lead to unexpected application faults. Notably, many unexpected faults localized to specific cores are due to silent data corruptions in the hardware layer. Consequently, developers often invest significant time debugging code when SDCs are the actual issue, as noted in hyperscale application studies [45], [32], [23]. In our fleet, SDCs affect computations involving various data types, including floating point, integers, vectors, mixed precision, special function units, data transfers, compression, and cryptography. Servers with SDCs significantly impact application health and can appear as software faults.

We structure events within the kernel execution context, categorizing workload failures by execution core and application logging events. This involves categorizing application type, offending subroutine, and execution frequency. By evaluating application misbehavior alongside fleet telemetry on a server, we identify hardware faults, specifically silent data corruptions. Integrating structured application failure data with fleet-wide datasets enhances SDC detection at scale.

3 Environment Setup

The experiment was conducted on the entire hyperscale server fleet. This fleet comprises several million servers, and consists of multiple generations of hardware.

3.1 Provisioning Systems

Servers in the infrastructure require a provisioning system to configure machines for application use [97] [37]. Scheduled maintenance events [96] are conducted at scale for firmware upgrades and hardware servicing. Robust server monitoring is enabled through health telemetry features across software and hardware layers [18].

3.2 Health and Telemetry Systems

Health telemetry covers software aspects like performance and power profiling [94], application trace monitoring [104], code-path execution evaluation [102], caching performance [72], service response times [39], and application stalls [27]. At the hardware layer, it includes device performance counters, machine check registers, system logs, and server-board telemetry [58]. A baseboard management controller (BMC) [5] captures sensor and thermal data [40]. Distributed binaries or daemons execute frequently based on sampling intervals [25], sending data for aggregation, deduplication, and storage [38]. Detection services evaluate telemetry events for remediation actions [63], [65]. When a server is faulty, remediation actions [99] include application restarts, device restarts, reimages, or component swaps based on alert severity [57]. Telemetry data is archived in advanced data infrastructures [20] with caches [33], databases [22] [26] for querying [82] and indexing [57], [93]. Retrieval mechanisms include counter-based, text-based, query-based, and graphical dashboards [9], with configurable options for privacy, retrieval, and storage.

3.3 Application Deployment and Resource Management Systems

In this infrastructure, production applications execute according to application scheduler policies [53]. Applications are containerized or run natively based on requirements and capacity allocation policies [90], [83]. New software, telemetry agents, and kernels are tested on a subset of the infrastructure to assess stability based on application failure rate tolerances [70]. After initial monitoring, applications are slowly rolled out fleet-wide in stages. Failures are logged and archived for analysis [80]. Roll-outs have time-bound

expectations to enhance developer efficacy [35]. Prioritizing safe, on-schedule deployments and reducing unexpected rollout faults is crucial [103], [47], [34].

HWSentinel’s study environment includes applications running on infrastructure with these capabilities. Hardware and software health telemetry databases are essential for enabling HWSentinel.

3.4 Fleetscanner and Ripple

To detect and mitigate silent data corruptions, several fleet-testing techniques have been proposed [30]. The infrastructure implements capabilities like *Fleetscanner* and *Ripple*. Fleetscanner, an out-of-production testing approach, schedules tests during maintenance phases like repairs and firmware upgrades to detect data corruptions. Ripple, an in-production approach, co-locates tests with production workloads to detect corruptions concurrently. Fleetscanner’s testing time is in minutes, achieving one iteration across all servers in approximately six months. Ripple executes tests in milliseconds, co-locating with workloads via a scheduler. In the results section, we compare the efficacy of HWSentinel framework with implementations of Fleetscanner and Ripple across workloads, architectures and data center regions.

3.5 Design Trade-offs

We explore the trade-offs of testing-based approaches and the HWSentinel framework, focusing on performance, test coverage, and infrastructure costs. Out-of-production testing (like Fleetscanner) incurs maintenance overhead, requiring efficient execution of high-detection tests within time constraints. In-production testing (like Ripple) is unsuitable for customer environments or performance-sensitive applications. Both methods can be integrated into operational steps but are limited by test budgets and state space coverage. A *pass* result doesn’t ensure complete functional coverage or fault-free chips.

HWSentinel’s efficacy relies on granular traces of application failures and the ability to query and correlate data across large databases, adding instrumentation overhead. As an evolutionary approach, HWSentinel requires feedback on database selection and evaluation pipeline configurations to improve its accuracy. HWSentinel operates offline, detecting faults independently of application and infrastructure operations.

The framework described in this paper is productionized and operational. HWSentinel gathers data and detects outliers on a continuous basis. HWSentinel is agnostic to hardware and software configurations. Servers suspected of SDCs are aggregated and moved into a quarantine pool where they are preserved until the root-cause is identified.

4 Hardware Sentinel Framework

This section provides a detailed overview of the HWSentinel implementation. HWSentinel’s architecture comprises two primary components: the data plane and the control plane. **Data Plane:** This component encompasses all relevant databases queried during the evaluation process. **Control Plane:** This component offers configurable options that enable fine-tuning the selection criteria for optimal detection rates. All control plane selections are based on analytical observations. In subsequent sections of the paper, we walk through data blocks associated with the data plane, and available configuration options in the control plane. HWSentinel identifies either a faulty server or a faulty workload.

4.1 Data Plane

The HWSentinel data plane encompasses all data sources used in the evaluation process. The main focus of the data plane is to identify and extract the right data sources to aid the decision making process. In large scale infrastructure, there is a large volume of telemetry data with different retention policies. Our selection is based on relevance of the data set for detection, and the impact to the results discussed. We continuously track for false positives and false negatives to update the schema. Addition and removal of new and existing sources is evaluated continuously based on their signal to noise ratio in detecting SDCs. The data plane consists of the following sources: 1. Kernel Logs 2. Reboot Database 3. System Failure Log Database 4. SDC Test Result Databases 5. Repair Database 6. Core Concentration Metadata

4.1.1 Unstructured to structured kernel logs. A hyper-scale fleet runs multiple versions of kernel at any given point of time. Kernel function logs are extracted from an unstructured log aggregation and stored in a database, as shown in Figure 1. To facilitate SDC detection, kernel function exception logs are restructured into a standardized format, as depicted in Figure 2. This *structured* kernel log enables a monitoring methodology for determining whether workloads are healthy within a live fleet when executing within kernel context. All kernel exceptions like panics, lockups, General Protection Faults (gpf), Machine Check Exceptions (MCE), divide errors, stack corruptions, are categorized based on the defined schema. This results in a holistic data set that captures the behavior of misbehaving application routines across all workloads.

4.1.2 Reboot Database. Large-scale maintenance operations often require multiple reboots. However, unplanned reboots indicate that the system has become inoperable due to some underlying issue. Healthy servers typically experience infrequent reboots, minimizing application disruption. We collect reboot data from all servers in the fleet over a configurable time window. If a reboot is consistently accompanied by a crash dump with sufficient telemetry, hardware

Field Name	Description
Backtrace	Trace of instructions calls leading up to the exception
Category Key	Exception type along with main part of the backtrace, used to categorize application crashes
Hardware Clues	Extraction from dmesg that contains hardware error messages around the time of exception
Core	File path to core file on host
Exception Core	CPU Core that exception occurred on
Exception CPU	Physical CPU that exception occurred on
Exception Die	Physical CPU die exception occurred on (for CPUs with chiplets)
Exception Module	Kmod is a multi-call binary that implements programs used to control kernel modules, reports the kmod that was running during exception time
Exception Socket	Socket the CPU that experienced an exception was in
Exception Workload/Task	Task running at the time of exception
Exception Handler	Reports information about the dying process kernel message log, uses ptrace to query the process memory image generator
Exception Type	Type of exception reported by the CPU that led to the application failure
Container Jobs	Container jobs running on the server at time of failure (gathered from local container state on disk available after reboot)

Figure 2. Structured Log Format

health check services can remediate the server [57]. Frequent reboot data sets without related fault telemetry have served as a good heuristic data set for the HWSentinel evaluation flow, and in plugging existing infrastructure remediation gaps as well as in detecting SDCs.

4.1.3 System Failure Log Database. Out-of-band mechanisms like the BMC log a subset of hardware faults in the System Event Log (SEL) based on firmware configurations. The SEL entries contain telemetry data such as power events, hardware errors, and environmental sensor readings. Data is captured as a binary stream but can be viewed in text format through decoders, and subsequently logged in databases for all servers for further analysis.

4.1.4 SDC Test Result Databases. We use databases of prior SDC detections from similar implementations (Fleetscanner and Ripple) to validate our results and identify historical fault patterns. Because each test execution has limited coverage, applications may experience SDCs before the tests

ultimately detect the faulty device. These databases have a longer retention policy than others, which helps with validation. If a server experiences application-level exceptions without a prior SDC detection in our databases, it suggests a higher likelihood of SDCs being present. We use these SDC test result databases as baselines for evaluating the effectiveness of HWSentinel compared to testing-based approaches like Fleetscanner and Ripple.

4.1.5 Repair Database. When existing telemetry diagnoses a faulty server in the fleet, the tooling system then identifies the specific faulty component and generates a repair ticket for data center technicians to replace it. All the repair actions in the fleet are logged.

4.1.6 Core Concentration Metadata. In a live fleet, hundreds of thousands of exceptions can occur across millions of servers daily, spanning multiple workloads. Even with structured logs, the logs present a large volume of data to parse through to create an eligible subset for evaluation. A single binary failing on an entire fleet of servers due to a software error can immediately result in at least a few million log entries in the database. As a result, it is easy to exhaust the database with false positive entries. Using core concentration metadata helps reduce false positives by focusing on high-value signals. We mark a log entry as valid when multiple workloads have exhibited failures on the same server. When multiple workloads fail on the same core or sibling cores, it's a strong indication of a hardware problem. Additionally, if one workload has failed across a large population of servers at the same backtrace and exception handler, it is likely a software fault. However, if the pattern persists and more workloads fail with the same backtrace across a longer time horizon, the failures increase in value as an evaluation signal. A static rule set isn't sufficient to capture all the nuances associated with fault correlation, so the policies in this block require ongoing attention and evaluation to eliminate false positives. Our framework prioritizes machines with multiple workload failures that exhibit core recidivism or exceptions in sibling cores. We capture backtrace recidivism across longer time horizons and increasing failure populations at the same backtrace across workloads as high value data sources.

4.2 Control Plane

Control plane hosts all the configuration parameters for HWSentinel. Figure 3 illustrates the overall flow for evaluating SDCs using both the data plane and control plane.

4.2.1 Reboot Configuration Parameters. The reboot database has an expansive history of reboots over a period of six months encountered by servers in the fleet. We utilize two configurable parameters for this database. The first parameter is the look-back window. This determines the length of the time window to search through for reboots. The second

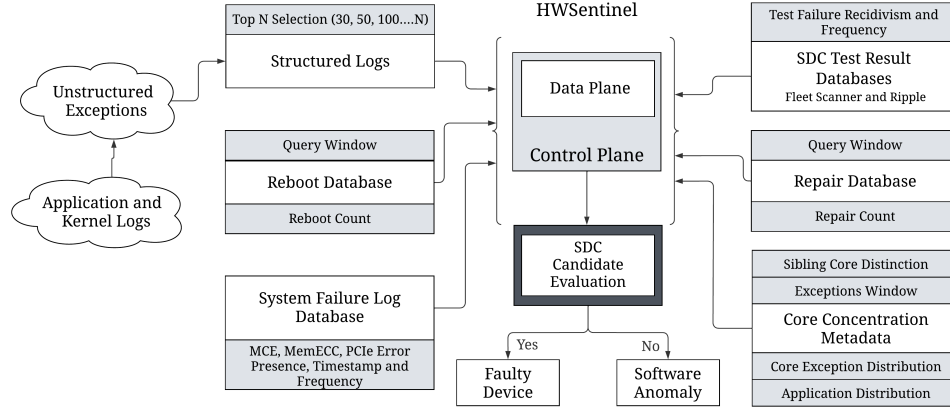


Figure 3. HWSentinel SDC Evaluation Flow

parameter is the number of reboots encountered within the specified look-back window. The current look-back window for the HWSentinel Flow is set to *30 days* and balances data accuracy with the cost of data retention and query load. The current threshold for the general server fleet is set to *6 reboots* within *30 days*, based on the average number of unexpected reboots determined from the reboot data set. For the AI fleet, the impact of a reboot is higher, especially during training workloads as multiple servers are halted due to a rebooting device. Due to tighter requirements on AI fleet availability, the threshold is set to *3 reboots* within *30 days*. If a server equals or exceeds the threshold defined, it is investigated for issues symptomatic of historical SDCs.

4.2.2 SEL Configuration Parameters. The framework checks for specific entries in the SEL at the time of the application failures. The search space is tuned for errors pertaining to memory ECC, MCE, and PCIe, thermal and sensor events. There is a large volume of published literature and standards describing these errors [48], [13], [2], [4]. Errors in these categories strongly indicate a faulty hardware component on the server. The search hypothesis with HWSentinel is that the instability of workloads is due to an *SDC inducing device*. As a result, only servers having **zero** SEL entries indicating hardware fault for the CPU are considered eligible for SDC detection pipeline. If the SEL telemetry indicates a CPU fault, the fault is *loud* and not silent. SEL entries are searched at similar timestamps as application exceptions during kernel routines. The absence of telemetry available in the SEL at a similar timestamp for CPU-related faults is a key consideration for categorization, indicating a device exhibiting a silent behavior while applications are encountering SDC failures.

4.2.3 Core Concentration Configuration. When a workload experiences an exception in kernel space, the responsible CPU sockets and cores are recorded. Single-core failures

are a key indicator of faulty CPUs, as applications typically utilize multiple CPU cores. If multiple applications fail on the same CPU and core, it strongly suggests that the CPU is the root cause of the problem. For the core concentration metadata field, numerous thresholds are set up, which we detail below:

Exceptions window. We set the look-back window to one week. Our analysis shows that a one-week look-back period strikes a good balance between accuracy and granularity. Additionally, selecting a one-week look-back window takes into account the infrastructure costs associated with higher granularity storage at fleet scale.

Core Exception Distribution. We aggregate all application exceptions per server and calculate their dispersion across each core on the CPU on a weekly basis. If a single core is responsible for more than half of the exceptions across all workloads, it is flagged as suspicious. The threshold for the percentage of exceptions on a single core is set to *60%* based on analytical observations, allowing for tuning based on workload sensitivity and depth of application telemetry. We observed that this configuration provided a good balance between false positives and detecting SDCs.

Sibling Core Distinction. Exceptions occurring on both the logical and physical cores of the CPU are considered independently in an attempt to understand per core exceptions. Early results with this policy led to a lower detection rate. However, when hyperthreaded or sibling cores are aggregated together, the detection rate is higher than evaluating individual cores.

Hyperthreaded cores largely share the same physical hardware [62]. As a result, aggregating workload exceptions across sibling cores as a heuristic increased detection efficacy. All of the *core concentration metadata* is evaluated in aggregate with sibling cores, and their respective exception counts. The *60%* threshold is applied on both single core and

Data Plane												
Hostname	Server Model Type	Top N Selection	Reboot Database	System Failure Log Database			SDC Test Results Databases: Fleetscanner & Ripple	Repair Database	Core Concentration Metadata			Faulty CPU or Software Anomaly?
			Exceeds Threshold?	PCIe Errors?	MCE?	ECC?	Have any hardware checks/tests failed?	Number of Swaps?	Core Exception Distribution	Sibling Core Distribution	Application Distribution	
Control Plane												
Host A	Model A	Yes	Yes	No	No	No	No	1	exceeds 60%	exceeds 60%	across multiple applications	Faulty CPU
Host B	Model A	Yes	No	No	No	No	No	0	does not exceed	does not exceed	not across multiple applications	Software Anomaly
Host N	Model B	Yes	Yes	No	No	Yes (Memory but not CPU)	No	2	exceeds 60%	exceeds 60%	across multiple applications	Faulty CPU

Figure 4. HWSentinel reference output table (data and control planes)

aggregated sibling core samples. If the values exceed the threshold, the CPU is marked as faulty.

Application Distribution. An important aspect of the core distribution threshold is the nature of application distribution across all cores. An application threshold is set based on the number of unique applications which have encountered exceptions on the same core or the aggregated sibling cores. If only one application has encountered a high failure rate across many servers on a particular core, the signal is considered to be low confidence and mostly due to software failure. Our current fleet-wide threshold is 2. If at least two applications have failed on the same set of sibling cores or single cores, the 60% threshold described above is triggered. However, if the distribution of a single application continues to increase with time across many servers, it is given special consideration as a potential unique SDC inducing workload. It is our observation that this is rare, as common libraries and modules are shared across all workloads. Our next control configuration addresses this scenario partly.

4.2.4 Top N Selection. We prioritize isolating servers with recurring exceptions to focus on persistent issues rather than transient errors. While the thresholds described in the previous section help reduce the state space of servers eligible for the pipeline, an evaluation is conducted for the *top N* (with *N* being configurable) candidate servers solely based on the number of exceptions. This evaluation is workload-agnostic and allows devices with the most samples to consistently rise to the top, giving weight to repeated patterns, especially those related to application or core distribution. Even if other thresholds are applied, repeated appearances in the *top N* list indicate a potential coverage gap in the HWSentinel Flow, prompting a review of control plane settings. We experiment for discovering best-fit for each workload for the appropriate *N*. For enterprise workloads, this parameter would need to be tuned based on the internal setup.

4.2.5 Repair Database. We use a configurable look-back window to analyze the repair database. Repairs are classified into three categories: misdiagnosed, undiagnosed, or correctly diagnosed. Correctly diagnosed repairs accurately target the problematic component and apply the appropriate remediation. Misdiagnosed repairs target either the wrong part or apply the wrong remediation or both. Undiagnosed repairs lack a diagnosis for the fault and requires manual intervention. The framework checks for device component swap history associated with the repair. A server with no prior repair history but a high number of workload exceptions and no known hardware issues is a prime candidate. A server that is repeatedly misdiagnosed or undiagnosed has a higher probability for SDC occurrences. Since the process of repair can take days depending on operational targets, identifying recidivism due to a lack of diagnosis or misdiagnosis requires a relatively longer time window. Through our analysis of the repair database, we determine that a 30-day time window is sufficient to confidently attribute recidivism to SDC.

4.2.6 SDC Test Result Database Controls - Fleetscanner and Ripple. Servers flagged by Ripple or Fleetscanner are considered as baseline for SDC detection. HWSentinel’s detection efficacy is validated by comparing its performance with that of hardware tests like Ripple and Fleetscanner. For the evaluation of the HWSentinel framework, servers with hits in the SDC database are excluded as detection candidates identified by HWSentinel. Furthermore, servers flagged by HWSentinel but not by Ripple or Fleetscanner indicate areas of improvement in the hardware tests.

The two planes together lead to the final HWSentinel schema with reference entries in Figure 4 with evaluation concluding to a faulty CPU or a software anomaly.

4.2.7 Selection Window Sensitivity. We previously discussed various selection windows and configurations in sections 4.2.1, 4.2.2, 4.2.3, and 4.2.5, including lookback windows,

number of reboots and core concentrations. These values were chosen based on two factors: data availability and detection accuracy. The 30 day lookback windows discussed in sections 4.2.1 and 4.2.5 were based on the retention of data sources, the data plane extracts the reboot and repair information from. A machine that experiences SDCs will continuously do so and exhibit unhealthy hardware behavior like rebooting, and consistently going through repair without resolution. A consistent pattern of behavior over 30 days increases our confidence in identifying a potential SDC candidate. Reducing this time frame would decrease the number of data points available in repair and reboot data sources, increasing the likelihood of capturing transient errors. This reduction in data introduces the chance that we will capture transient errors and incorrectly label them as hardware problems.

The number of reboots within the 30 day lookback window referenced in section 4.2.1 was chosen based off what we qualify as a healthy system. In an ideal production environment unexpected reboots infrequently occur. Six reboots within a 30-day window indicate frequent disruptions, occurring approximately once or twice a week for four consecutive weeks. Increasing this threshold (from six) would allow for more frequent service interruptions, negatively impacting fleet availability. For AI workloads, we reduce the threshold to three reboots due to stricter availability requirements.

In section 4.2.3, assigning a core exception distribution of 60% indicates that if more than half of all exceptions occur on one CPU core, it is likely indication of malfunctioning hardware. Lowering this threshold to 50% or less would suggest that multiple cores are experiencing similar exception rates, making it more likely that the application is causing the issue.

4.3 Practical Examples

This section presents two case studies that illustrate the HWSentinel Flow. We follow a host through the HWSentinel data and control plane to demonstrate the significance of the thresholds described in 4.1 and 4.2. Subsequently, describe false positive identification, mentioned in 4.1.

4.3.1 Example Selection Flow. In this section we go through the HWSentinel data and control plane to exhibit the significance of the thresholds described in 4.1 and 4.2. The host was selected for evaluation through the Top N Selection process (see 4.2.4). This host was experiencing large numbers of exceptions periodically which made it a good candidate for SDC investigation. Digging a level deeper into the flow, the host had experienced 7 reboots in the last 30 days, exceeding our threshold defined in section 4.2.1. Based on this information, we initially suspect that the host may have a faulty CPU. However, after evaluating our core concentration parameter (see 4.2.3), we found that the exceptions were evenly distributed across all cores, rather than being

concentrated (with a concentration threshold of 60%) on a single core. Since the exceptions were not concentrated on a single core, we conclude that the host is more likely to have a *software anomaly* rather than a *faulty CPU*. When multiple CPU cores experience the same level of exceptions at once, it is more likely that the application running on the cores caused the exception. It is very rare that multiple cores on the same processor would fail due to hardware reasons without triggering any other errors in the fleet.

4.4 Example False Positive Flow

During the development of HWSentinel's data and control plane, we initially focused on identifying applications that crashed at a higher rate than others. We then investigated the specific hosts on which these applications crashed. HWSentinel flagged several hosts as containing a *Faulty CPU* due to their unhealthy behavior, which exceeded the thresholds set by the control plane. Further investigation revealed that the application was designed to intentionally crash the system. This was a fuzzer tool, and the crashing behavior was expected; however, HWSentinel incorrectly attributed the issue to the hardware, resulting in a false positive (see 4.1). To prevent similar false positives, we have modified our approach to exclude fuzzer workloads from our analysis.

5 Results

Since its deployment, we have compared the detection efficacy of HWSentinel with that of FleetScanner and Ripple, two existing fleet testing mechanisms.

5.1 Metrics for Comparison

We present the results as a relative detection rate, which represents the number of detections made by HWSentinel across the population tested. We calculated the relative detection rate (RDR) for HWSentinel in comparison to FleetScanner and Ripple. The RDR is calculated using the following formula:

$$RDR = \frac{HWSentinelDetectionRate}{FleetScannerDetectionRate + RippleDetectionRate}$$

where

$$DetectionRate = \frac{DetectedFaultyCPUs}{TotalCPUsScreened}$$

We illustrate HWSentinel's relative detection ratio across CPU generations, different workload families, and regions. Our results show improvements in identifying SDCs across all categories with HWSentinel.

5.2 Architectures

We evaluate 8 distinct CPU generations from various families. The CPUs in this study are manufactured between 2012 to 2023. These CPU generations are designated as SC-1 to SC-8. We observe that inclusion of HWSentinel improves

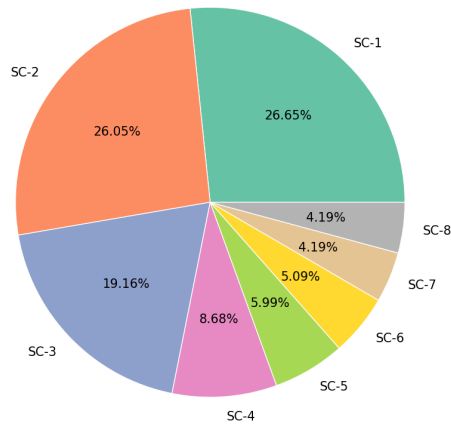


Figure 5. HWSentinel CPU Architecture Detection Break-down

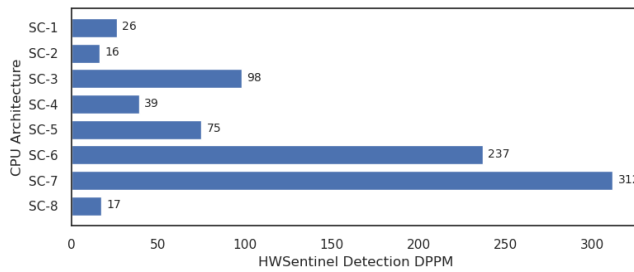


Figure 6. HWSentinel CPU Architecture Unique DPPM relative to fleet

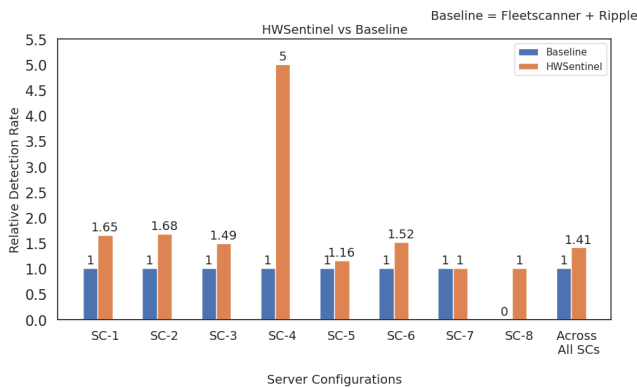


Figure 7. HWSentinel Detection in CPU Architectures vs Testing Mechanisms Combined

the detection of SDCs in all but the SC-7 configuration. SC-7 represents a CPU architecture in early adoption phases with negligible volume of workloads executing on the architecture. This lack of improvement is due to the absence of significant computational load in the applications running

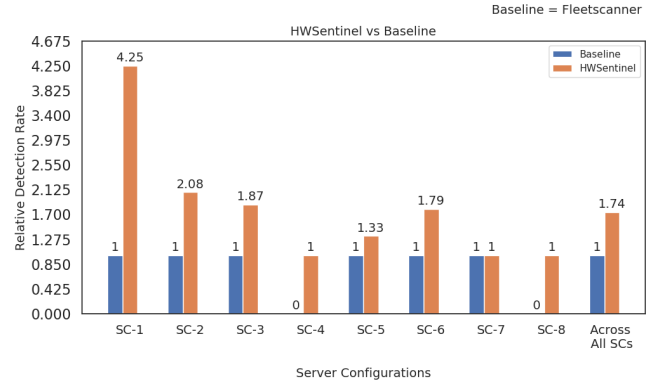


Figure 8. HWSentinel Detection in CPU Architectures vs Fleetscanner-like Testing Methods

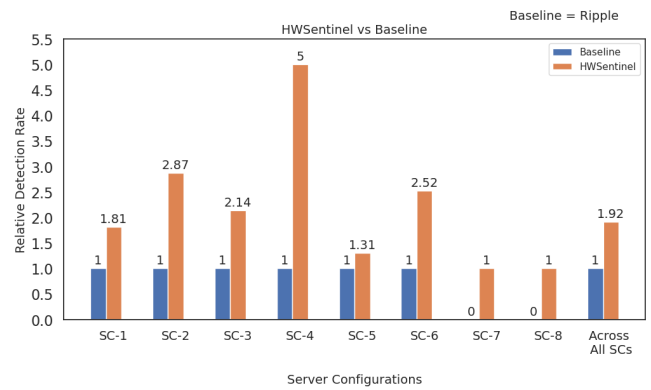


Figure 9. HWSentinel Detection in CPU Architectures vs Ripple-like Testing Methods

on this architecture. Our results demonstrate that HWSentinel scales across all CPU generations and architectures, as shown in Figure 5. Figure 6 presents the DPPM values for each CPU architecture, calculated using HWSentinel and compared to the total number of CPUs in each architecture. This DPPM is significantly higher than what we had detected using typical techniques for the same detection period.

Coverage from Ripple and Fleetscanner is used as the baseline for SDC coverage efficacy. In comparison to individual coverage for Fleetscanner, addition of HWSentinel improves coverage by $1.31x$ to $2.87x$ in all but one generation. For SC-4 and SC-8 we notice that Fleetscanner fails to detect any of the SDCs that HWSentinel detects; showcasing the inefficacy of tests on selected architectures. HWSentinel performs even better when compared individually with Ripple. Improvements to fleet coverage with HWSentinel ranges between $1.31x$ to $5x$. For SC-4 we notice the highest improvement; $5x$ in comparison to the baseline coverage because the architecture and associated workloads have the lowest cadence of testing opportunities in comparison to the other architectures within the fleet, which results in testing approaches

requiring a longer time to reach comparable detection. SC-4 also predominantly serves Edge and Gaming workloads which do not yield higher detection even with co-location of in-production tests due to the performance sensitivity associated with the workloads. For SC-7 and SC-8 Ripple failed to detect the SDCs that HWSentinel did. This is due to the tests being non-conductive to be co-located with workloads on these architectures. Tests for SC-7 and SC-8 show lower defect coverage overall. For SC-8 specifically, both the existing test mechanisms fail to detect any SDCs, and we are reliant on HWSentinel for detection. SC-8 dedicated tests have the lowest testing efficacy across the entire fleet, specifically due to run-time versus coverage inefficiencies.

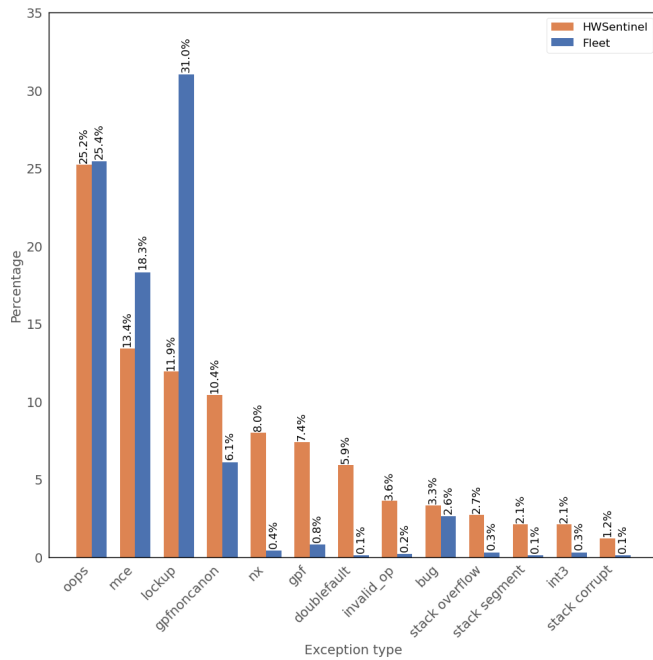


Figure 10. HWSentinel Detection across Exception Types vs. Fleet-wide detection

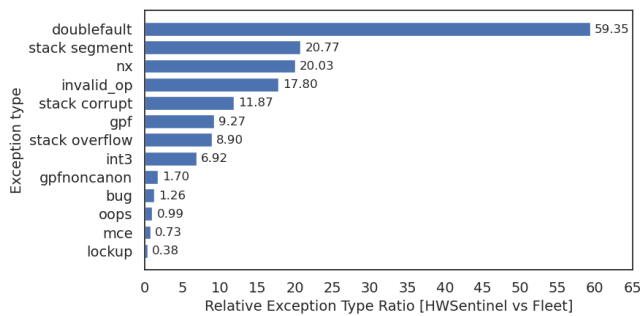


Figure 11. Relative Exception Type Ratio [HWSentinel vs Fleet]

5.3 Exception Types

Figure 10 presents the frequency of exception types detected by Hardware Sentinel and compares it to the fleet-wide frequency. Some exception types, such as *lockups* and *oops*, are common indicators of software anomalies and are frequently observed in production environments. Others, like *stack segment*, *int3*, *doublefault*, *invalid ops*, and *nx*, are less common in the fleet but show a notable presence in HWSentinel positives. To illustrate the difference in frequency, we calculate a ratio between HWSentinel’s detection rate and the fleet-wide rate, as shown in Figure 11. This relative fleet exception ratio is calculated using the following formula:

$$RelativeExceptionTypeRatio = \frac{HWSentinelExceptionDetectionRate}{FleetwideExceptionDetectionRate}$$

We provide a brief description below of exceptions which correlate the most to hardware failures:

Stack Segment. A stack segment exception occurs when an operating system is trying to access the memory region allocated for a stack that is an invalid address within the segment stack [21].

Int3. A Breakpoint exception occurs at the execution of the INT3 instruction. A debug software can replace an instruction by the INT3 instruction [21].

Doublefault. A double fault can occur when an exception is unhandled or when the CPU tries to call an exception handler but fails. A double fault is a serious kernel misbehavior that cannot be recovered. The faulting process must be terminated [21].

Invalid op. The Invalid Opcode exception occurs when the processor tries to execute an invalid or undefined opcode, or an instruction with invalid prefixes. It can occur if the instruction tries to access a non-existent control register [21].

Nx. An NX Linux kernel exception occurs when an attempt is made to execute machine code in a memory region that has been marked as non-executable [21].

We observe a significantly higher occurrence of exceptions within HWSentinel detections compared to the fleet, with increases of **20.77x**, **6.92x**, **59.35x**, **17.80x**, and **20.03x** for *stack segment*, *int3*, *double fault*, *invalid op*, and *nx* exceptions, respectively. These rare exception types aggregated on one core are a strong indicator for faulty CPUs, made apparent by their increased presence in hosts identified by HWSentinel. **High prevalence of *stack segment*, *int3*, *doublefault*, *invalid op* and *nx* located on one core are novel indicators of an SDC inducing CPU, uniquely observed from Hardware Sentinel.** This categorization has not been previously identified by any other SDC detection frameworks.

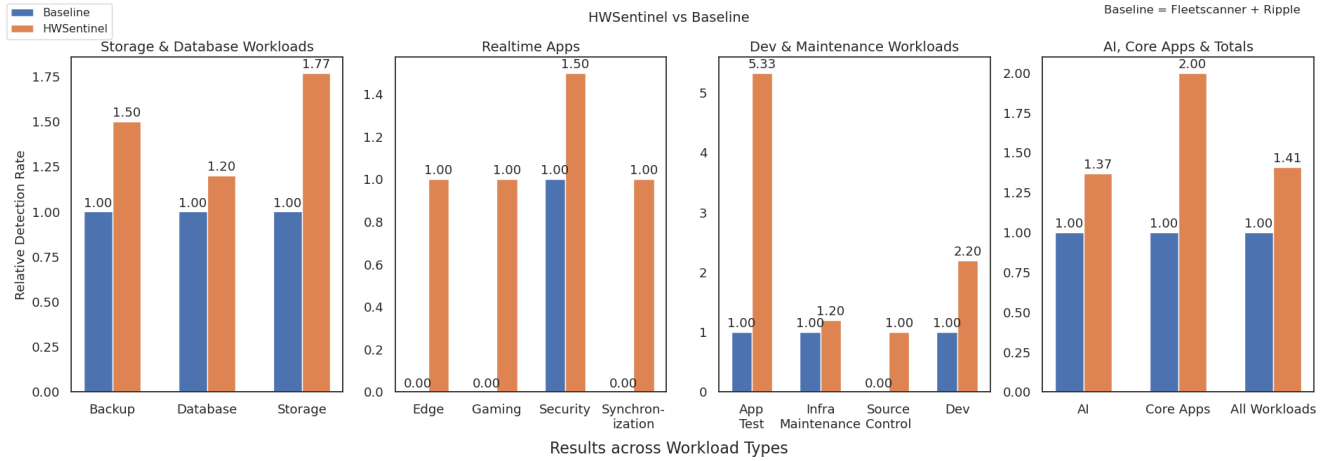


Figure 12. HWSentinel Detection across workloads vs Testing mechanisms combined

5.4 Workloads

We categorize workloads into 13 distinct groups, reflecting the diversity of fleet-wide workloads. At a high level, we group workloads into four families: *Storage and Database Workloads*, *Realtime Applications*, *Developer and Maintenance Workloads*, and *AI and Core Applications*. The lowest improvement of 1.2x is observed in *Infra Maintenance* workloads over combined testing approaches, while the highest improvement of 2.2x is observed in *Dev* workloads. *Infra Maintenance* workloads are primarily schedule-driven, enabling more frequent testing via Fleetscanner. *Infra Maintenance* workloads show minimal benefits with HWSentinel. The results can be broadly attributed to the frequency at which these workloads execute the methods and data structures we log and analyze. In contrast, *AppTest* and *Dev* workloads experience heavier loads and greater code heterogeneity, leading to improved detection rates with HWSentinel. *AppTest*, benefits from its synthetic and diverse testing sequences, which enhance detection rates. *Dev* workloads also benefit from consistent code and library changes, as well as the inherent heterogeneity of developmental workloads, leading to better detection rates with HWSentinel. Storage workloads operate with relatively lower capacity machine pools and have tighter capacity requirements and longer time penalty associated with draining and undraining of services due to large-storage architectures. Lower capacity pools lead to higher contention among workloads and the testing frequency under maintenances to be lower. Thus for *Backup* and *Storage* workloads, the detection efficacy of HWSentinel shows the largest improvement.

HWSentinel is the sole approach that successfully detects SDCs in workloads such as *Edge*, *Gaming*, *Source Control*, and *Synchronization*.

In workloads like *Edge*, *Gaming*, *Source Control* and *Synchronization*, HWSentinel is the only approach that successfully detected SDCs. These workloads do not follow traditional maintenance schedules and execute tests less frequently via opportunistic methods like Fleetscanner, making HWSentinel’s approach more effective. While these workloads also executed Ripple, the coverage was not as effective as other workloads due to performance sensitivity and programming constructs that lead to lower efficacy of Ripple executions. However, with HWSentinel’s approach we bypassed these challenges and purely relied on code path exceptions and derivative log analytics to detect SDCs.

5.5 Data Center Regions

We evaluate the efficacy of HWSentinel across 27 data center regions. Fleetscanner and similar approaches tend to favor certain regions due to prioritized maintenance schedules based on server age and data center design. Even though the entire fleet obtained coverage over a six month period, certain regions would run tests far more frequently than others. HWSentinel’s methods are region agnostic and require understanding of disparate logs across databases. Our results show that HWSentinel significantly enhances fleet coverage in 23 out of 27 regions, as depicted in Figure 13. In newer regions like *region7* and *region13* HWSentinel provides maximum benefits due to the initial convergence time required for Fleetscanner and Ripple frameworks. However, in regions 17, 18, 19, and 20, the addition of HWSentinel does not provide significant benefits compared to the combined testing approaches. This is because these regions frequently execute tests and predominantly house *SC5* architectures with high test efficacy, resulting in minimal benefits from HWSentinel. We expect that with time and the dynamic nature of workloads and novel architectures, all regions will show improvement for HWSentinel. In summary, HWSentinel increases SDC coverage by 41% (or 1.41 times) when added to

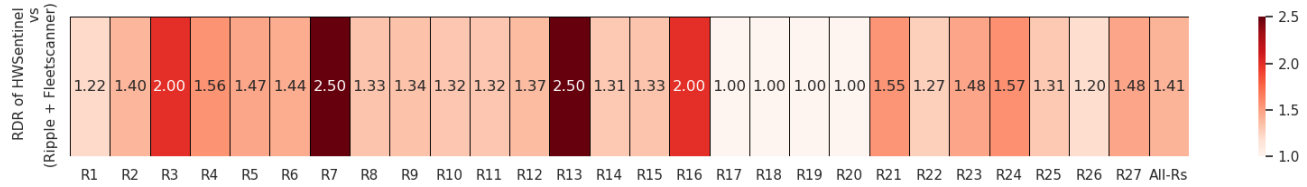


Figure 13. Relative Detection Rate of HWSentinel Detection across Regions vs Baseline (Fleetscanner + Ripple)

the fleet, surpassing the combined coverage of Fleetscanner and Ripple.

5.6 Kernel Rollouts

HWSentinel enhances workload availability and accelerates code deployment. We observe a significant reduction in kernel test exceptions before and after the framework went live, demonstrating the significant positive impact of removing SDC-affected servers from rollout pools on application health. Before HWSentinel, approximately **30%** of kernel exceptions were linked to the *top 100 CPU core-server-region* candidate samples, which consistently encountered SDCs and were undetected by FleetScanner and Ripple. After introducing HWSentinel, we observe that *only 9%* of the kernel exceptions are attributed to top 100 CPU core-server-region samples and that the majority of the remaining exceptions are software related. The consistent elimination of candidate samples with confirmation of SDCs from testing pool improves the workload rollout process. The percentage of exceptions continues to decrease with improved detection and categorization across hardware and software root-causes. HWSentinel resulted in a **21% reduction in exceptions** across the fleet by identifying and removing corrupt servers undetectable by previous health remediation tools.

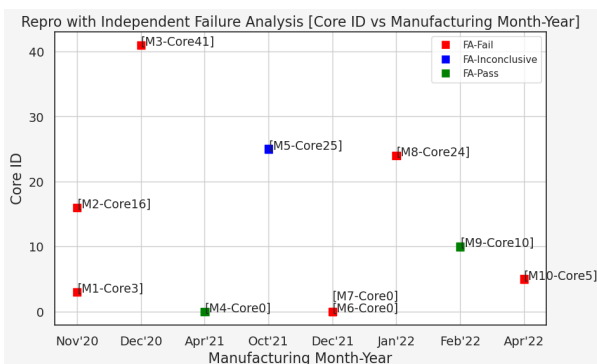


Figure 14. HWSentinel Failure Reproduction via independent failure analysis

5.7 Independent Validation of Results

We present the results of an independent, third-party failure analysis conducted on a random selection of 10 samples detected as faulty by HWSentinel in Figure 14. The samples were from different data center regions. The CPU generation selection was at the guidance of the third party. The samples observed failures across different cores and manufacturing months. After the independent third party failure analysis, 7 out of the 10 samples conclusively reproduced the SDCs, with 1 sample being inconclusive, indicating that the HWSentinel is not only valuable for infrastructure detection but also for sample extraction for semiconductor failure analysis. Reproduction rates for SDCs are typically low, ranging from 15% to 30% [101], [85], [75]. In contrast, HWSentinel achieved a significantly higher failure reproduction rate of 70%, specifically for SDC-causing devices, which are notoriously difficult to detect and reproduce. Furthermore, the samples detected by HWSentinel also failed when subjected to more extensive SDC detection tests, which involved longer durations and larger seed pools, which are infeasible to execute at fleet scale. The reproducibility of SDCs is challenging due to the complex state space, varying application codes, denser chips and smaller technology nodes, making it even more impressive that HWSentinel was able to achieve such a high failure reproduction rate. The reproducibility validates the HWSentinel framework as effective in detecting SDC inducing chips with independent failure analysis. We continue to work with semiconductor manufacturers and share data-driven insights to plug test gaps, develop new tests and aid debug and failure analysis with additional supporting logs.

5.8 Discussion

HWSentinel offers scalable and agnostic detection capabilities across various workloads, regions, and architectures. It provides significant improvements in coverage compared to existing methods, including a 74% increase over Fleetscanner, a 92% increase over Ripple, and a 41% increase over the combined efficacy of both methods. When combined, Fleetscanner and Ripple form a superset of SDC detection, but HWSentinel still outperforms this superset by 41%. Each approach offers distinct yet complementary coverage, enhancing our understanding of SDCs in the fleet. The state space of detected SDCs and understanding of corruptions

at scale is enhanced with these approaches. HWSentinel enables academic researchers to explore SDC resilience without relying on proprietary tools or vendor-specific information.

Our results demonstrate that SDCs can lead to significant issues in modern fleets, leading to eventual consistency and accuracy implications. Moreover, their occurrence rate is orders of magnitude higher and results in greater impact than cache coherency issues in modern fleets. For instance:

- In CPU architectures *SC-1* and *SC-2*, SDCs caused locks not to be shared between threads, resulting in consistency issues.
- In *SC-3*, corrupted cache-line valid bits led to repeated reads of invalidated data into the application, masquerading as coherency issues.

These defects masqueraded as consistency and coherency issues but originated silently in hardware. We plan to present additional case studies in future work.

6 Related Work

Silent data corruptions (SDCs) are a critical issue in hyperscaler fleets. Meta’s case study [31] presented a comprehensive analysis of a production failure, including reverse engineering and mitigation strategies. Google’s paper [45] on SDCs discussed corrupt execution errors and faulty cores, validating Meta’s findings on occurrence rates. Meta’s *RocksDB* [32] and Google’s *Spanner* [23] papers overview SDC challenges in large applications. Meta’s *Detecting SDCs in the wild* [30] highlights Fleetscanner and Ripple testing costs. Alibaba [100] used vendor tools for deeper characterization, corroborating other hyperscalers. AMD categorized SDC definitions [87], and chip manufacturers have open-sourced test frameworks [43], [55], [61].

Research on soft error-induced data corruptions has a long history, with notable contributions from Mukherjee et al. [66], [67]. Further investigations followed into voltage scaling impacts [11] and CPU/DRAM effects [64], [59]. Recent research addresses SDC challenges from semiconductor defects. Trippel [8] discussed SDC evolution and challenges. Papadimitriou et al. [73], [74] focused on SDC modeling in microarchitectural blocks using gem5 simulators [19]. Singh et al. [84] explored detection approaches for microarchitectural SDC aspects. Harpocrates [51] achieved shorter test generation times. Jafarzadeh et al. [49] worked on root-causing SDCs with pattern generation. Serebryany et al. [81] developed fuzzing techniques for CPU corruption detection. DelayAVF [29] identified small delay faults as SDC root causes. Rahman et al. [76] reduced test space in PEPPA-X. Mummidi et al. [69], [68] and Huang et al. [46] explored self-checking and self-healing. Nvidia’s ECC SwapCodes [88] combined instruction duplication with error coding. Unlike previous approaches, HWSentinel is designed to be hardware-agnostic, eliminating the need for hardware-specific details.

SDCs also impact AI workloads, as demonstrated by recent research on fault-tolerance in neural networks and DNN fault mitigation. Kosaian et al. [52] evaluated fault-tolerance in neural networks on GPUs. He et al. [44] and Li et al. [56] explored DNN fault mitigation. Ares introduced a fault injection framework for DNN resilience [78]. Dr. DNA studied neuron activation distribution to reduce SDCs in DNNs [60]. With LLMs, SDC resilience is a key research topic, as discussed by Google’s Gemini [92] and Agarwal et al. [10]. This list highlights the growing interest in SDCs across industry, academia, and media.

7 Conclusion

In this paper, we present HWSentinel, a framework for detecting silent data corruptions (SDCs) at the application level. We capture comprehensive framework requirements, including detailed architecture descriptions, structured logging schemas, data and control plane requirements. We demonstrate the effectiveness of HWSentinel in a hyperscale fleet across 8 CPU generations, 13 workload families, and 27 data center regions, and identify top exceptions correlated with SDCs. Additionally, HWSentinel enables more reliable software releases and kernel deployments. We validate the framework through independent failure analysis of randomly selected components, achieving significantly higher reproducibility. Through novel reverse engineering and large-scale analytics techniques, we identify previously undetected hardware faults. HWSentinel enhances server reliability and performance while minimizing the risk of application failures due to SDCs.

Acknowledgements The authors would like to extend special thanks to Arushi Sharma and Leandro Silva for their invaluable assistance in enabling the infrastructure tooling aspects of the flow. Thanks to Tejasvi Chakravarthy, Matt Beadon, Daniel Moore, Corentin Cos and the rest of the infrastructure engineers for their support and guidance in enabling this flow within our infrastructure. We would like to express our gratitude to our vendor partners for facilitating FA requests and supporting end-to-end validation of the HWSentinel flow.

References

- [1] Interrupt Descriptor Table - IDT - OSDev Wiki. https://wiki.osdev.org/Interrupt_Descriptor_Table.
- [2] PCISIG specifications. <https://pcisig.com/specifications>. Accessed: 2024-09-30.
- [3] System Calls- OSDev Wiki. <https://wiki.osdev.org/SystemCalls>. Accessed: 2024-09-30.
- [4] DDR4 JEDEC Standard Specification. <https://www.jedec.org>, Jul 2021.
- [5] OpenBMC. <https://www.openbmc.org/>, June 2023.
- [6] Computing’s Hidden Menace: The OCP Takes Action Against Silent Data Corruption (SDC). <https://www.opencompute.org/blog/computings-hidden-menace-the-ocp-takes-action-against-silent-data-corruption-sdc>, Jan 2024.

- [7] Open Compute Project. <https://www.opencompute.org/>, Jun 2024.
- [8] Caroline Trippel on Aug 15. Silent data corruption at scale, Aug 2022.
- [9] Lior Abraham, John Allen, Oleksandr Barykin, Vinayak Borkar, Bhuwan Chopra, Ciprian Gerea, Daniel Merl, Josh Metzler, David Reiss, Subbu Subramanian, et al. Scuba: Diving into data at facebook. *Proceedings of the VLDB Endowment*, 6(11):1057–1067, 2013.
- [10] U. Agarwal, A. Chan, and K. Pattabiraman. Resilience assessment of large language models under transient hardware faults. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pages 659–670, Los Alamitos, CA, USA, oct 2023. IEEE Computer Society.
- [11] Dimitris Agiakatsikas, George Papadimitriou, Vasileios Karakostas, Dimitris Gizopoulos, Mihalis Psarakis, Camille Bélanger-Champagne, and Ewart Blackmore. Impact of voltage scaling on soft errors susceptibility of multicore server cpus. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 957–971, 2023.
- [12] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. A scalable, commodity data center network architecture. *ACM SIGCOMM computer communication review*, 38(4):63–74, 2008.
- [13] AMD. AMD Documentation Hub.
- [14] Lakshmi N. Bairavasundaram, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, Garth R. Goodson, and Bianca Schroeder. An analysis of data corruption in the storage stack. *ACM Trans. Storage*, 4(3), nov 2008.
- [15] Luiz André Barroso, Jeffrey Dean, and Urs Holzle. Web search for a planet: The google cluster architecture. *IEEE micro*, 23(2):22–28, 2003.
- [16] R.C. Baumann. Radiation-induced soft errors in advanced semiconductor technologies. *IEEE Transactions on Device and Materials Reliability*, 5(3):305–316, 2005.
- [17] Eduardo Berrocal, Leonardo Bautista-Gomez, Sheng Di, Zhiling Lan, and Franck Cappello. Lightweight silent data corruption detection based on runtime data analysis for hpc applications. In *Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing*, pages 275–278, 2015.
- [18] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site reliability engineering: How Google runs production systems*. "O'Reilly Media, Inc.", 2016.
- [19] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, et al. The gem5 simulator. *ACM SIGARCH computer architecture news*, 39(2):1–7, 2011.
- [20] Dhruba Borthakur, Jonathan Gray, Joydeep Sen Sarma, Kannan Muthukkaruppan, Nicolas Spiegelberg, Hairong Kuang, Karthik Ranganathan, Dmytro Molkov, Aravind Menon, Samuel Rash, Rodrigo Schmidt, and Amitanand Aiyer. Apache hadoop goes realtime at facebook. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*, SIGMOD '11, page 1071–1080, New York, NY, USA, 2011. Association for Computing Machinery.
- [21] Daniel P. Bovet and Marco Cesati. *Understanding the linux kernel*, third edition. 2005.
- [22] Guoqiang Jerry Chen, Janet L Wiener, Shridhar Iyer, Anshul Jaiswal, Ran Lei, Nikhil Simha, Wei Wang, Kevin Wilfong, Tim Williamson, and Serhat Yilmaz. Realtime data processing at facebook. In *Proceedings of the 2016 International Conference on Management of Data*, pages 1087–1098, 2016.
- [23] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson C. Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. Spanner: Google's globally distributed database. *ACM Trans. Comput. Syst.*, 31:8, 2013.
- [24] Victor Costan and Srinivas Devadas. Intel sgx explained. *Cryptology ePrint Archive*, 2016.
- [25] Brian Coutinho. Dynolog: Open source system observability, Nov 2022.
- [26] Sudipto Das, Miroslav Grbic, Igor Ilic, Isidora Jovandic, Andrija Jovanovic, Vivek R Narasayya, Miodrag Radulovic, Maja Stikic, Gaoxiang Xu, and Surajit Chaudhuri. Automatically indexing millions of databases in microsoft azure sql database. In *Proceedings of the 2019 International Conference on Management of Data*, pages 666–679, 2019.
- [27] Harish Dattatraya Dixit, Fan Lin, Bill Holland, Matt Beadon, Zhengyu Yang, and Sriram Sankar. Optimizing interrupt handling performance for memory failures in large scale data centers. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, pages 193–201, 2020.
- [28] Jeffrey Dean. Large-scale distributed systems at google: Current systems and future directions. In *The 3rd ACM SIGOPS International Workshop on Large Scale Distributed Systems and Middleware (LADIS 2009) Tutorial*, 2009.
- [29] Peter W. Deutsch, Vincent Quentin Ulitzsch, Sudhanva Gurumurthi, Vilas Sridharan, Joel S. Emer, and Mengjia Yan. Delayavf: Calculating architectural vulnerability factors for delay faults. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 231–245, 2024.
- [30] Harish Dattatraya Dixit, Laura Boyle, Gautham Vunnam, Sneha Pendharkar, Matt Beadon, and Sriram Sankar. Detecting silent data corruptions in the wild. *arXiv preprint arXiv:2203.08989*, 2022.
- [31] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. Silent data corruptions at scale. *arXiv preprint arXiv:2102.11245*, 2021.
- [32] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications. *ACM Trans. Storage*, 17(4), oct 2021.
- [33] Hamed Esfahani, Jonas Fietz, Qi Ke, Alexei Kolomiets, Erica Lan, Erik Mavrinac, Wolfram Schulte, Newton Sanches, and Srikanth Kandula. Cloudbuild: Microsoft's distributed and caching build service. In *Proceedings of the 38th International Conference on Software Engineering Companion*, pages 11–20, 2016.
- [34] Aleksander Fabijan, Pavel Dmitriev, Helena Holmström Olsson, and Jan Bosch. The benefits of controlled experimentation at scale. In *2017 43rd Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 18–26, 2017.
- [35] Dror G Feitelson, Eitan Frachtenberg, and Kent L Beck. Development and deployment at facebook. *IEEE Internet Computing*, 17(4):8–17, 2013.
- [36] David Fiala, Frank Mueller, Christian Engelmann, Rolf Riesen, Kurt Ferreira, and Ron Brightwell. Detection and correction of silent data corruption for large-scale high-performance computing. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–12, 2012.
- [37] Anshul Gandhi, Mor Harchol-Balter, Ram Raghunathan, and Michael A Kozuch. Autoscale: Dynamic, robust capacity management for multi-tier data centers. *ACM Transactions on Computer Systems (TOCS)*, 30(4):1–26, 2012.
- [38] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The google file system. In *Proceedings of the nineteenth ACM symposium on Operating systems principles*, pages 29–43, 2003.
- [39] Rahul Ghosh, Kishor S Trivedi, Vijay K Naik, and Dong Seong Kim. End-to-end performability analysis for infrastructure-as-a-service cloud: An interacting stochastic models approach. In *2010 IEEE 16th Pacific rim international symposium on dependable computing*, pages 125–132. IEEE, 2010.
- [40] Glaucio Gonçalves, Daniel Rosendo, Leylane Ferreira, Guto Leoni Santos, Demis Gomes, Andre Moreira, Judith Kelner, Djamel Sadok,

- Mattias Wildeman, and Patricia Takako Endo. A standard to rule them all: Redfish. *IEEE Communications Standards Magazine*, 3(2):36–43, 2019.
- [41] Weining Gu, Zbigniew Kalbarczyk, Ravishankar K Iyer, and Zhenyu Yang. Characterization of linux kernel behavior under errors. In *2003 International Conference on Dependable Systems and Networks, 2003. Proceedings.*, pages 459–459. IEEE Computer Society, 2003.
- [42] Siva Kumar Sastry Hari, Sarita V. Adve, and Helia Naeimi. Low-cost program-level detectors for reducing silent data corruptions. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2012)*, pages 1–12, 2012.
- [43] Siva Kumar Sastry Hari, Paolo Rech, Timothy Tsai, Mark Stephenson, Arslan Zulfiqar, Michael Sullivan, Philip Shirvani, Paul Racunas, Joel Emer, and Stephen W Keckler. Estimating silent data corruption rates using a two-level model. *arXiv preprint arXiv:2005.01445*, 2020.
- [44] Yi He, Mike Hutton, Steven Chan, Robert De Gruijl, Rama Govindaraju, Nishant Patil, and Yanjing Li. Understanding and mitigating hardware failures in deep learning training systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA '23*, New York, NY, USA, 2023. Association for Computing Machinery.
- [45] Peter H. Hochschild, Paul Jack Turner, Jeffrey C. Mogul, Rama Krishna Govindaraju, Parthasarathy Ranganathan, David E Culler, and Amin Vahdat. Cores that don't count. In *Proc. 18th Workshop on Hot Topics in Operating Systems (HotOS 2021)*, 2021.
- [46] Yafan Huang, Shengjian Guo, Sheng Di, Guanpeng Li, and Franck Cappello. Mitigating silent data corruptions in hpc applications across multiple program inputs. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14, 2022.
- [47] Jez Humble and David Farley. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education, 2010.
- [48] Intel. Intel® 64 and ia-32 architectures software developer manuals.
- [49] Hanieh Jafarzadeh, Florian Klemme, Jan Dennis Reimer, Zahra Paria Najafi-Haghi, Hussam Amrouch, Sybille Hellebrand, and Hans-Joachim Wunderlich. Robust pattern generation for small delay faults under process variations. In *2023 IEEE International Test Conference (ITC)*, pages 111–116, 2023.
- [50] P. Jogalekar and M. Woodside. Evaluating the scalability of distributed systems. *IEEE Transactions on Parallel and Distributed Systems*, 11(6):589–603, 2000.
- [51] Nikos Karystinos, Odysseas Chatzopoulos, George-Marios Fragkoulis, George Papadimitriou, Dimitris Gizopoulos, and Sudhanva Gurumurthi. Harpocrates: Breaking the silence of cpu faults through hardware-in-the-loop program generation.
- [52] Jack Kosaian and K. V. Rashmi. Arithmetic-intensity-guided fault tolerance for neural network inference on gpus. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21*, New York, NY, USA, 2021. Association for Computing Machinery.
- [53] Kien Le, Ricardo Bianchini, Margaret Martonosi, and Thu D Nguyen. Cost-and energy-aware load distribution across data centers. *Proceedings of HotPower*, 1, 2009.
- [54] Gunho Lee, Byung-Gon Chun, and H. Katz. Heterogeneity-aware resource allocation and scheduling in the cloud. In *Proceedings of the 3rd USENIX Conference on Hot Topics in Cloud Computing, HotCloud'11*, page 4, USA, 2011. USENIX Association.
- [55] David P. Lerner, Benson Inkley, Shubhada H. Sahasrabudhe, Ethan Hansen, Luis D. Rojas Munoz, and Arjan van de Ven. Optimization of tests for managing silicon defects in data centers. In *2022 IEEE International Test Conference (ITC)*, pages 578–582, 2022.
- [56] Sihuan Li, Jianyu Huang, Ping Tak Peter Tang, Daya Khudia, Jongsoo Park, Harish Dattatraya Dixit, and Zizhong Chen. Efficient soft-error detection for low-precision deep learning recommendation models. In *2022 IEEE International Conference on Big Data (Big Data)*, pages 1556–1563, 2022.
- [57] Fan Lin, Matt Beadon, Harish Dattatraya Dixit, Gautham Vunnam, Amol Desai, and Sriram Sankar. Hardware remediation at scale. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, pages 14–17, 2018.
- [58] Fred Lin, Antonio Davoli, Imran Akbar, Sukumar Kalmanje, Leandro Silva, John Stamford, Yanai Golany, Jim Piazza, and Sriram Sankar. Predicting remediations for hardware failures in large-scale datacenters. In *2020 50th Annual IEEE-IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S)*, pages 13–16. IEEE, 2020.
- [59] Lucas Matana Luza, Daniel Söderström, Helmut Puchner, Rubén García Alía, Manon Letiche, Alberto Bosio, and Luigi Dilillo. Effects of thermal neutron irradiation on a self-refresh dram. In *2020 15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS)*, pages 1–6. IEEE, 2020.
- [60] Dongning Ma, Fred Lin, Alban Desmaison, Joel Coburn, Daniel Moore, Sriram Sankar, and Xun Jiao. Dr. dna: Combating silent data corruptions in deep learning using distribution of neuron activations. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS '24*, page 239–252, New York, NY, USA, 2024. Association for Computing Machinery.
- [61] Thiago Jose Macieira. Open data center diagnostic project. *Intel*.
- [62] Deborah T Marr, Frank Binns, David L Hill, Glenn Hinton, David A Koufaty, J Alan Miller, and Michael Upton. Hyper-threading technology architecture and microarchitecture. *Intel Technology Journal*, 6(1), 2002.
- [63] Justin Meza, Tianyin Xu, Kaushik Veeraraghavan, and Onur Muthu. A large scale study of data center network reliability. In *Proceedings of the Internet Measurement Conference 2018*, pages 393–407, 2018.
- [64] Sarah E Michalak, Kevin W Harris, Nicolas W Hengartner, Bruce E Takala, and Stephen A Wender. Predicting the number of fatal soft errors in los alamos national laboratory's asc q supercomputer. *IEEE Transactions on Device and Materials Reliability*, 5(3):329–335, 2005.
- [65] Justin Moore, Jeff Chase, Keith Farkas, and Parthasarathy Ranganathan. Data center workload monitoring, analysis, and emulation. In *Eighth workshop on computer architecture evaluation using commercial workloads*, pages 1–8, 2005.
- [66] Shubhendu S Mukherjee, Joel Emer, and Steven K Reinhardt. The soft error problem: An architectural perspective. In *11th International Symposium on High-Performance Computer Architecture*, pages 243–247. IEEE, 2005.
- [67] Shubu Mukherjee. *Architecture design for soft errors*. Morgan Kaufmann, 2011.
- [68] Chandra Sekhar Mummidi, Sandeep Bal, Brunno F. Goldstein, Sudarshan Srinivasan, and Sandip Kundu. A highly-efficient error detection technique for general matrix multiplication using tiled processing on simd architecture. In *2022 IEEE 40th International Conference on Computer Design (ICCD)*, pages 529–536, 2022.
- [69] Chandra Sekhar Mummidi, Victor C. Ferreira, Sudarshan Srinivasan, and Sandip Kundu. Highly efficient self-checking matrix multiplication on tiled amx accelerators. *ACM Trans. Archit. Code Optim.*, nov 2023. Just Accepted.
- [70] S Narayanan. The netflix tech blog: Preparing the netflix api for deployment, 2013.
- [71] Cliord Neuman. Scale in distributed systems. 01 2000.
- [72] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, et al. Scaling memcache at facebook. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, pages 385–398, 2013.

- [73] George Papadimitriou and Dimitris Gizopoulos. Silent data corruptions: Microarchitectural perspectives. *IEEE Transactions on Computers*, 72(11):3072–3085, 2023.
- [74] George Papadimitriou, Dimitris Gizopoulos, Harish Dattatraya Dixit, and Sriram Sankar. Silent data corruptions: The stealthy saboteurs of digital integrity. In *2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, pages 1–7, 2023.
- [75] Haiyu Qi, Sanka Ganesan, and Michael Pecht. No-fault-found and intermittent failures in electronic products. *Microelectronics Reliability*, 48(5):663–674, 2008.
- [76] Md Hasanur Rahman, Aabid Shamji, Shengjian Guo, and Guanpeng Li. Peppax: finding program test inputs to bound silent data corruption vulnerability in hpc applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21, New York, NY, USA, 2021. Association for Computing Machinery.
- [77] Layali Rashid, Karthik Pattabiraman, and Sathish Gopalakrishnan. Characterizing the impact of intermittent hardware faults on programs. *IEEE Transactions on Reliability*, 64(1):297–310, 2015.
- [78] Brandon Reagen, Udit Gupta, Lillian Pentecost, Paul Whatmough, Sae Kyu Lee, Niamh Mulholland, David Brooks, and Gu-Yeon Wei. Ares: a framework for quantifying the resilience of deep neural networks. In *Proceedings of the 55th Annual Design Automation Conference*, DAC '18, New York, NY, USA, 2018. Association for Computing Machinery.
- [79] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C. Snoeren. Inside the social network's (datacenter) network. *SIGCOMM Comput. Commun. Rev.*, 45(4):123–137, aug 2015.
- [80] Tony Savor, Mitchell Douglas, Michael Gentili, Laurie Williams, Kent Beck, and Michael Stumm. Continuous deployment at facebook and oanda. In *Proceedings of the 38th International Conference on software engineering companion*, pages 21–30, 2016.
- [81] Kostya Serebryany, Maxim Lifantsev, Konstantin Shtoyk, Doug Kwan, and Peter Hochschild. Silifuzz: Fuzzing cpus by proxy, 2021.
- [82] Raghav Sethi, Martin Traverso, Dain Sundstrom, David Phillips, Wenlei Xie, Yutian Sun, Nezih Yegitbasi, Haozhun Jin, Eric Hwang, Nileema Shingte, et al. Presto: Sql on everything. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1802–1813. IEEE, 2019.
- [83] Jay Shah and Dushyant Dubaria. Building modern clouds: Using docker, kubernetes google cloud platform. In *2019 IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0184–0189, 2019.
- [84] Adit Singh, Sreejit Chakravarty, George Papadimitriou, and Dimitris Gizopoulos. Silent data errors: Sources, detection, and modeling. In *2023 IEEE 41st VLSI Test Symposium (VTS)*, pages 1–12, 2023.
- [85] Peter Söderholm. A system view of the no fault found (nff) phenomenon. *Reliability Engineering & System Safety*, 92(1):1–14, 2007.
- [86] Vilas Sridharan, Nathan DeBardleben, Sean Blanchard, Kurt B. Ferreira, Jon Stearley, John Shalf, and Sudhanva Gurumurthi. Memory errors in modern systems: The good, the bad, and the ugly. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '15, page 297–310, New York, NY, USA, 2015. Association for Computing Machinery.
- [87] Gurumurthi Sudhanva, Sridharan Vilas, and Gurumurthy Sankar. Emerging fault modes: Challenges and research opportunities. 2023.
- [88] Michael B. Sullivan, Siva Kumar Sastry Hari, Brian Zimmer, Timothy Tsai, and Stephen W. Keckler. Swapcodes: Error codes for hardware-software cooperative gpu pipeline error detection. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 762–774, 2018.
- [89] Andrew S. Tanenbaum and Maarten Van Steen. *Distributed Systems: Principles and Paradigms*. Prentice Hall PTR, USA, 1st edition, 2001.
- [90] Chunqiang Tang, Kenny Yu, Kaushik Veeraraghavan, Jonathan Kaldor, Scott Michelson, Thawan Kooburat, Aravind Anbudurai, Matthew Clark, Kabir Gogia, Long Cheng, et al. Twine: A unified cluster management system for shared infrastructure. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 787–803, 2020.
- [91] Lingjia Tang, Jason Mars, Xiao Zhang, Robert Hagmann, Robert Hundt, and Eric Tune. Optimizing google's warehouse scale computers: The numa experience. In *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*, pages 188–197, 2013.
- [92] Gemini Team, Rohan Anil, Sebastian Borgeaud, et al. Gemini: A family of highly capable multimodal models, 2024.
- [93] Ashish Thusoo, Zheng Shao, Suresh Anthony, Dhruva Borthakur, Namit Jain, Joydeep Sen Sarma, Raghotham Murthy, and Hao Liu. Data warehousing and analytics infrastructure at facebook. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 1013–1020, 2010.
- [94] Kushagra Vaid. Datacenter power efficiency: Separating fact from fiction. In *Invited talk at the 2010 Workshop on Power Aware Computing and Systems*, volume 1, page 2, 2010.
- [95] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems*, EuroSys '15, New York, NY, USA, 2015. Association for Computing Machinery.
- [96] Saranyan A. Vignanam and Benjamin Leonhardi. Maintaining large-scale ai capacity at meta, June 2024.
- [97] Daniel Villela, Prashant Pradhan, and Dan Rubenstein. Provisioning servers in the application tier for e-commerce systems. *ACM Trans. Internet Technol.*, 7(1):7–es, feb 2007.
- [98] Lawrence C Wagner. *Failure analysis of integrated circuits: tools and techniques*, volume 494. Springer Science & Business Media, 1999.
- [99] Guosai Wang, Lifei Zhang, and Wei Xu. What can we learn from four years of data center hardware failures? In *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 25–36, 2017.
- [100] Shaobu Wang, Guangyan Zhang, Junyu Wei, Yang Wang, Jiesheng Wu, and Qingchao Luo. Understanding silent data corruptions in a large production cpu population. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 216–230, New York, NY, USA, 2023. Association for Computing Machinery.
- [101] R Williams, J Banner, I Knowles, M Dube, M Natishan, and M Pecht. An investigation of 'cannot duplicate' failures. *Quality and Reliability Engineering International*, 14(5):331–337, 1998.
- [102] Ran Xu, Subrata Mitra, Jason Rahman, Peter Bai, Bowen Zhou, Greg Bronevetsky, and Saurabh Bagchi. Pythia: Improving datacenter utilization via precise contention prediction for multiple co-located workloads. In *Proceedings of the 19th International Middleware Conference*, Middleware '18, page 146–160, New York, NY, USA, 2018. Association for Computing Machinery.
- [103] Ya Xu, Nanyu Chen, Addrian Fernandez, Omar Sinno, and Anmol Bhasin. From infrastructure to culture: A/b testing challenges in large scale social networks. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, pages 2227–2236, 2015.
- [104] Yibo Zhu, Nanxi Kang, Jiaxin Cao, Albert Greenberg, Guohan Lu, Ratul Mahajan, Dave Maltz, Lihua Yuan, Ming Zhang, Ben Y Zhao, et al. Packet-level telemetry in large datacenter networks. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pages 479–491, 2015.